Adapter Design Pattern

Prepared BY

Team The Axis (Team 6)

University of Barishal

Instructor

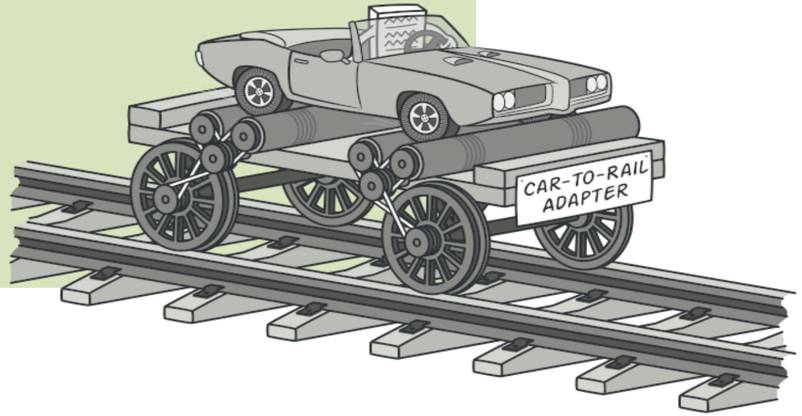
Md Samsuddoha

Assistant Professor
University of Barishal

What is the Adapter??

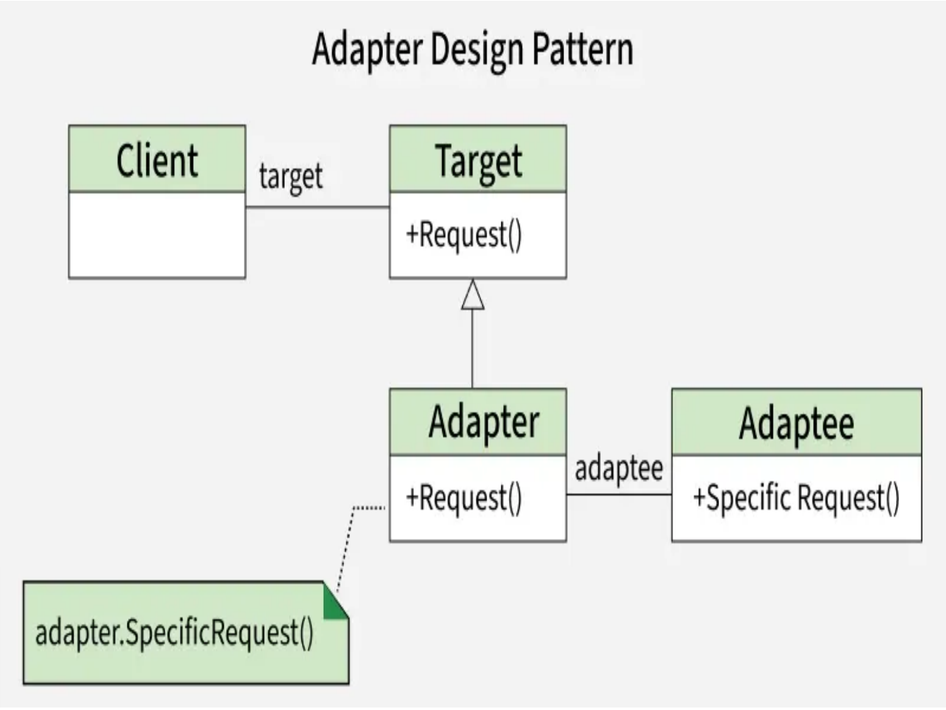
Adapter Design Pattern is a **structural pattern** that acts as a bridge between two incompatible interfaces, allowing them to work together.

- Integrate legacy code or third-party libraries into new systems
- Enable collaboration without modifying source code



In the this second Diagram

- Client wants to use a Target interface (it calls **Request()**).
- Adaptee already has useful functionality, but its method (SpecificRequest()) doesn't match the Target interface.
- Adapter acts as a bridge: it implements the Target interface (Request()), but inside, it calls the Adaptee's **SpecificRequest().**
- This allows the Client to use the Adaptee without changing its code.



Adapter Pattern — Why We Need It

1. **Incompatible interfaces**

   Lets two classes work together even though their interfaces don't match.

2. **Reuse existing code**

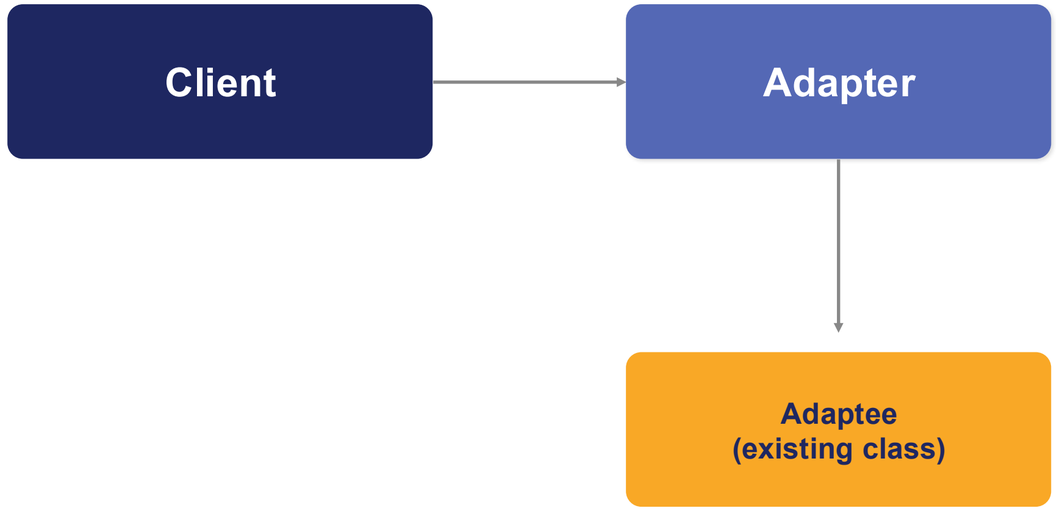
   Wrap legacy or third-party classes without modifying their source.

3. **Bridge between systems**

   Translates calls from the client's expected interface to the target's actual interface.

4. **Open/Closed principle**

   Add new compatibility without changing existing, tested code.



Caption: Client calls a familiar interface; the Adapter translates the call into the Adaptee's actual method.

Adapter — Real-Life Example

Travel Power Plug Adapter

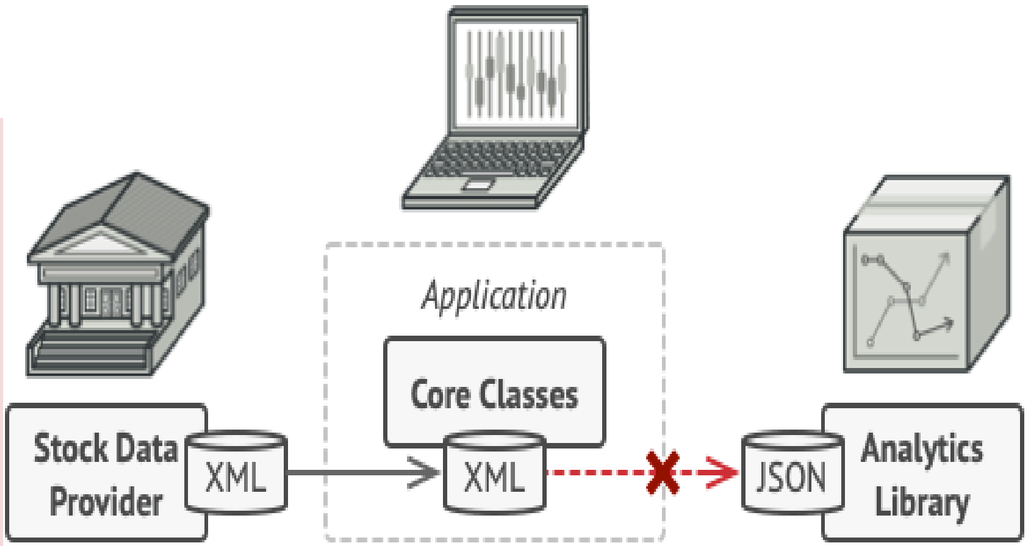
- A laptop has a US-style plug (Client expects a US socket).
- The wall socket in Europe is a different shape (Adaptee interface).
- A travel plug adapter sits between them, converting the US plug shape into one the EU socket accepts.
- Neither the laptop nor the wall socket is modified — the adapter bridges them.

Code parallel: EUSocketAdapter implements USPlugInterface, internally calling EUSocket.connect().

In software, an example is wrapping a JSON-only API with an Adapter so an app expecting XML can use it unchanged.

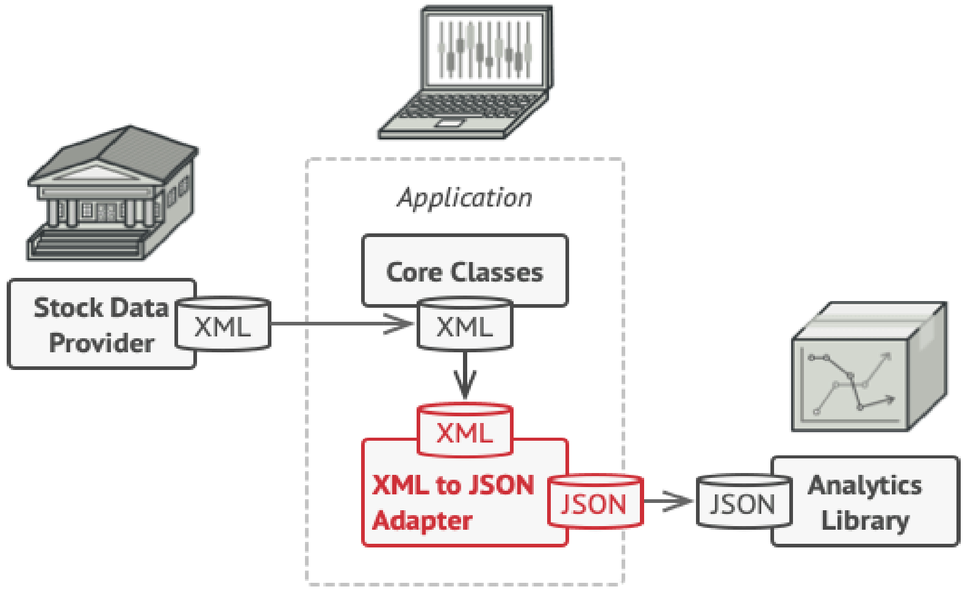
Problem

- The stock market app receives data in XML format.
- A third-party analytics library only accepts JSON format.
- The two systems have incompatible interfaces.
- Modifying the library may break existing code or may not be possible due to lack of source code.
- Therefore, the app and the analytics library cannot communicate directly.



Solution

- Create an Adapter object to bridge incompatible interfaces.
- The adapter converts requests and data into a format understood by the target object.
- It hides the complexity of conversion from both objects.
- Existing classes can interact through the adapter without modifying their source code.
- In the stock market app, XML-to-JSON adapters translate XML data into JSON before passing it to the analytics library.



Solution code:

```java
// Target Interface
interface MediaPlayer {
    void play(String audioType, String fileName);
}

// Adaptee Class
class Mp4Player {
    void playMp4(String fileName) {
        System.out.println("Playing MP4 file: " + fileName);
    }
}

// Adapter Class
class MediaAdapter implements MediaPlayer {
    private Mp4Player mp4Player = new Mp4Player();

    public void play(String audioType, String fileName) {
        if (audioType.equalsIgnoreCase(anotherString: "mp4")) {
            mp4Player.playMp4(fileName);
        }
    }
}

// Client Class
public class Main {
    public static void main(String[] args) {
        MediaPlayer player = new MediaAdapter();
        player.play(audioType: "mp4", fileName: "song.mp4");
    }
}
```

I. MediaPlayer → Target Interface
II. Mp4Player → Adaptee (existing incompatible class)

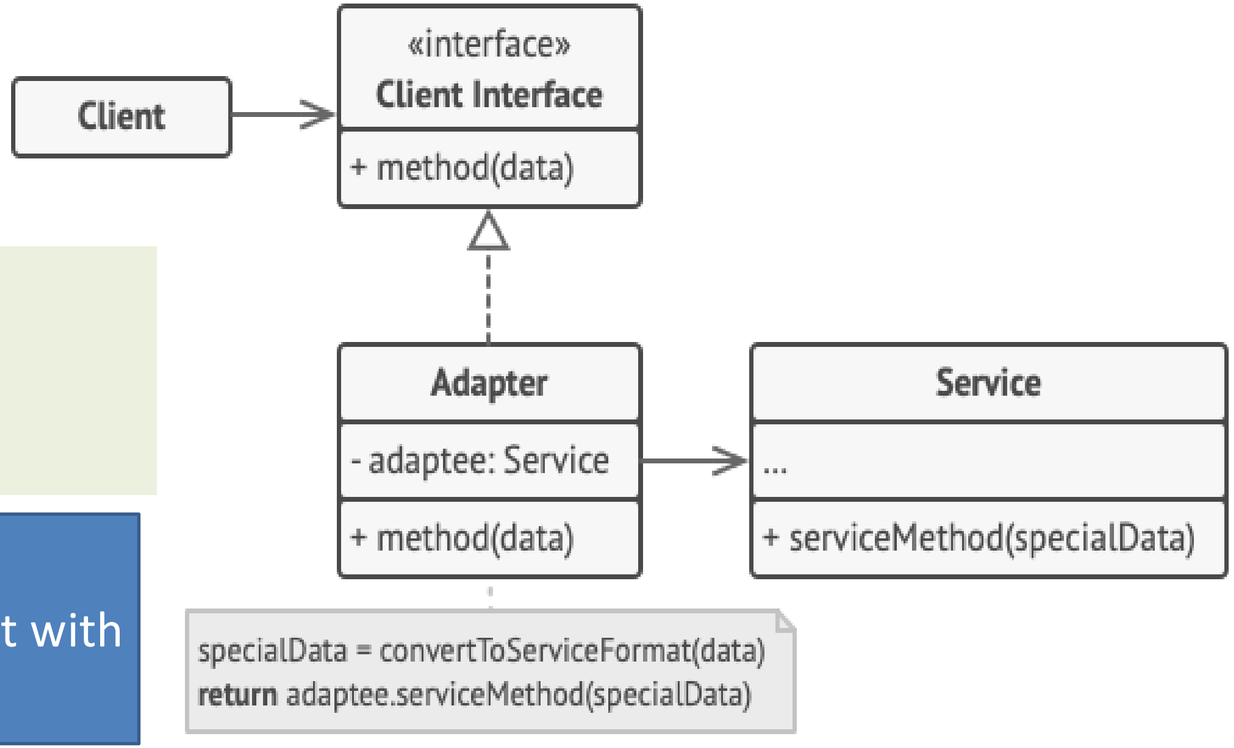
III. MediaAdapter → Adapter that connects the two
IV. Main → Client using the adapter

Output:
Playing MP4 file: song.mp4

Structure

Object adapter

- ❑ Uses composition.
- ❑ Wraps the adaptee object.
- ❑ Bridges incompatible interfaces.
- ❑ Supported by all major programming languages.



1.Client Interface
Defines the protocol that classes must follow to interact with the client.

2.Service (Adaptee)
•Existing or third-party class with an incompatible interface.

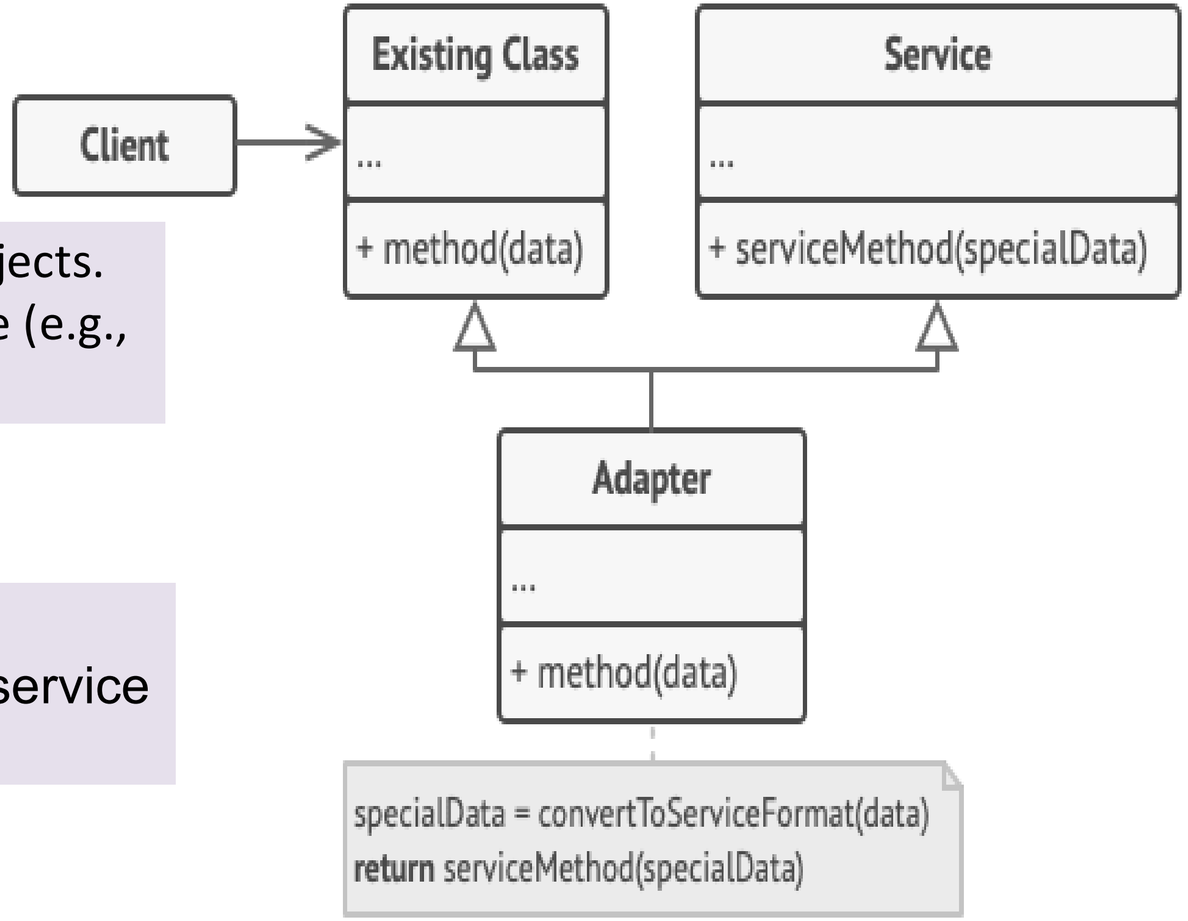
3.Adapter
- ➢ Implements the client interface and wraps the service object.
- ➢ Translates client requests into a format understood by the service.

4.Client
Communicates with the service through the adapter.
Remains independent of specific adapter implementations.

Class adapter

- Uses inheritance by inheriting interfaces from both objects.
- Requires a language that supports multiple inheritance (e.g., C++).

- Uses inheritance and does not wrap objects.
- Works by overriding methods to adapt client and service behavior.



Pros

1.Follows Single Responsibility Principle by separating conversion logic from business logic.
2.Supports Open/Closed Principle by allowing new adapters without changing existing code.

Cons:

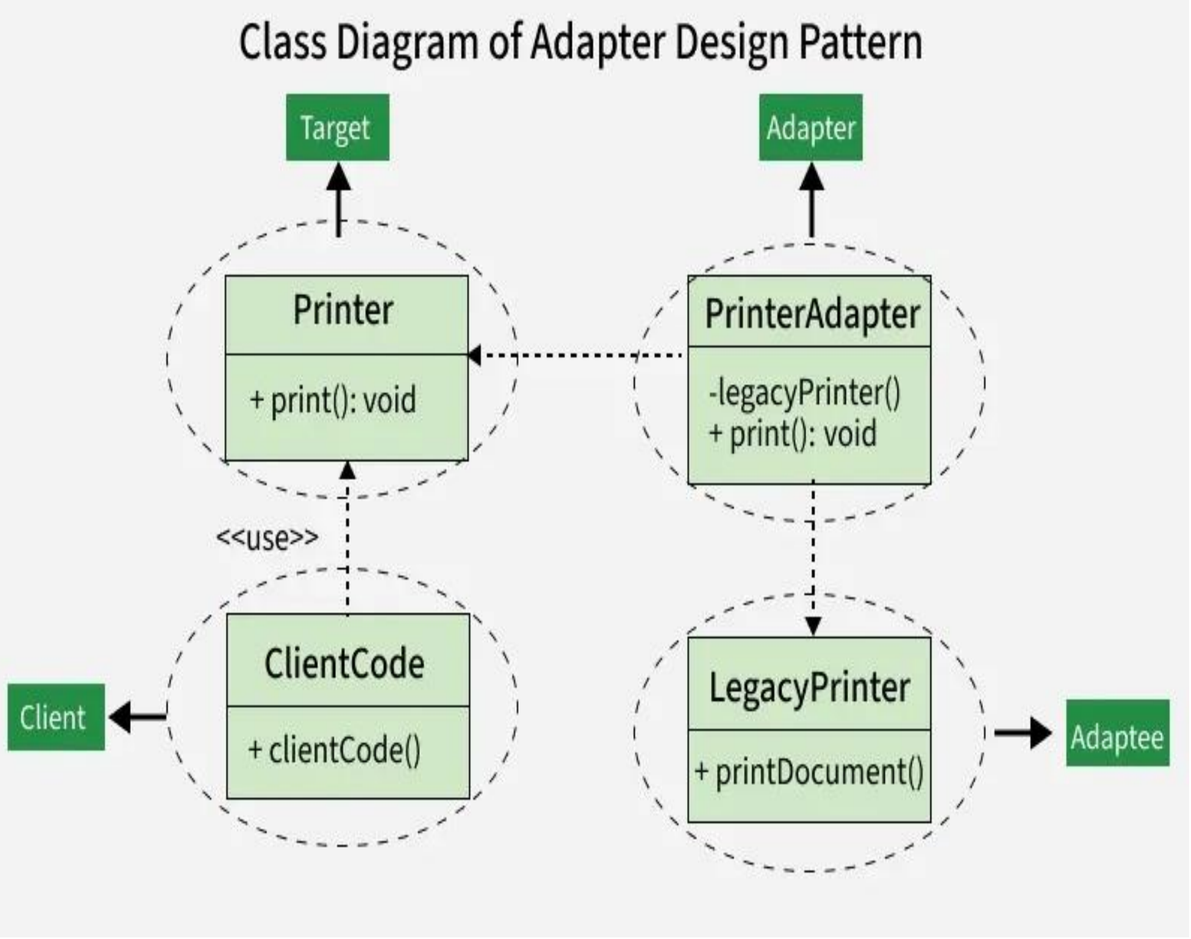
1. Increases overall code complexity due to additional interfaces and classes.
2. Sometimes modifying the service class is simpler than using an adapter.

When to Use Adapter Pattern

a. When you want to use an existing class with an incompatible interface.
b. When integrating legacy code or third-party libraries into your system.
c. When you need to reuse classes that cannot be modified directly.

Implementation Example

- The system uses a LegacyPrinter class with method printDocument(),
- The new system expects a Printer interface with method print() , making them incompatible.



1.Target Interface (Printer)

The interface that the client code expects.

```
/* Target Interface */
interface Printer {
    void print();
}
```

2. Adaptee (LegacyPrinter)

The existing class with an incompatible interface.

```
/* Adaptee */

class LegacyPrinter {
    public void printDocument() {
        System.out.println("Legacy Printer is printing a document.");
    }
}
```

3. Adapter (PrinterAdapter)

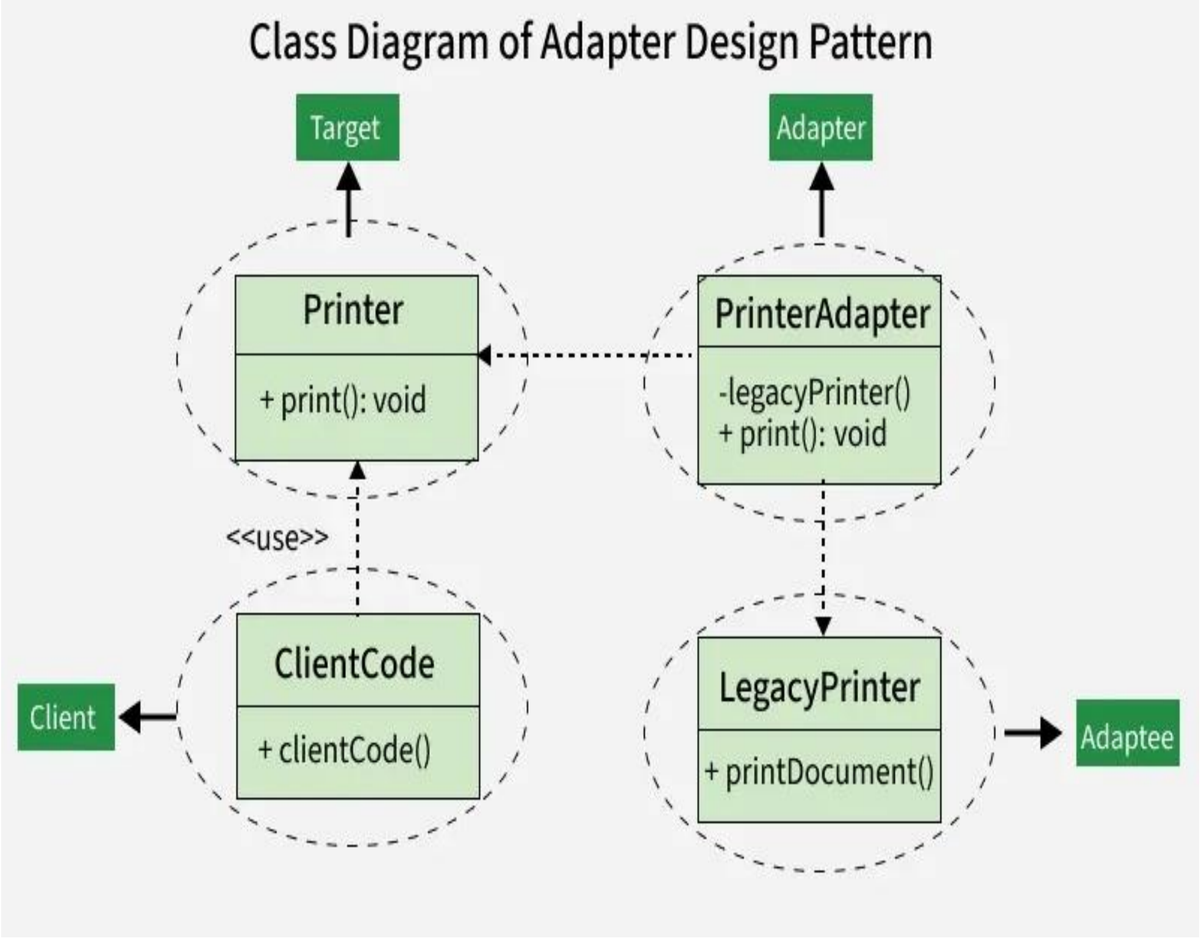
The class that adapts the LegacyPrinter to the Printer interface.

```java
/* Adapter */

class PrinterAdapter implements Printer {
    private LegacyPrinter legacyPrinter;

    public PrinterAdapter(LegacyPrinter legacyPrinter) {
        this.legacyPrinter = legacyPrinter;
    }

    @Override
    public void print() {
        legacyPrinter.printDocument();
    }
}
```



4. Client Code

The code that interacts with the Printer interface.

```java
/* Client Code */
interface Printer {
    void print();
}

void clientCode(Printer printer) {
    printer.print();
}
```

References

Refactoring Guru — Mediator
refactoring.guru/design-patterns/mediator (main reference: concept, structure, pros & cons)

GeeksForGeeks
Design Patterns: Elements of Reusable Object-Oriented Software (GoF, 1994) — original definition

Thank You

Any Questions?